SC4051 Distributed Systems

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28

Contents

Chapter 1

Foundations: Fallacies, Models and the CAP Landscape

> *"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable."* — Leslie Lamport. This chapter names the forces that make that quote true and gives you the vocabulary to reason about them.

From zero: no distributed-systems background assumed. We start from a single program on one machine and ask what changes when we add a network, a second machine, and the possibility that either or the network can fail. Every term is defined on first use. We move from intuition to formal models to the CAP theorem.

Learning Outcomes

After this chapter you will be able to:

(L1) list and explain the eight fallacies of distributed computing with concrete failure examples;

(L2) distinguish the synchronous, partially synchronous and asynchronous timing models;

(L3) define crash-stop, crash-recovery, omission and Byzantine failure models;

(L4) state and prove the CAP theorem and explain its practical scope;

(L5) classify a system (DNS, ZooKeeper, Dynamo) by its consistency–availability trade-off.

1.1 Why Distribution Is Different

A single machine is deceptively simple: memory is shared, failure is total (it crashes or it does not), and there is one clock. Add a network and all three assumptions break. The network can drop, delay, duplicate or reorder messages. Machines fail independently. Clocks drift apart. What looks like a procedure call becomes a protocol

with timeouts, retries and partial failure.

Peter Deutsch's *eight fallacies of distributed computing* catalog the dangerous assumptions:

1. The network is reliable.
2. Latency is zero.
3. Bandwidth is infinite.
4. The network is secure.
5. Topology does not change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

Each fallacy has bitten a real system. Fallacy 1: a mobile app that assumes TCP always delivers and loses orders on a tunnel handover. Fallacy 2: a chatty RPC loop that makes 100 sequential calls and turns 1 ms per call into 100 ms of user-visible latency. Fallacy 4: a flat internal network that assumed no attacker inside—until a compromised printer became a pivot.

The remedy is not to patch each fallacy but to design with them as premises: expect loss, account for latency, budget bandwidth, authenticate everything, handle reconfiguration, and embrace heterogeneity.

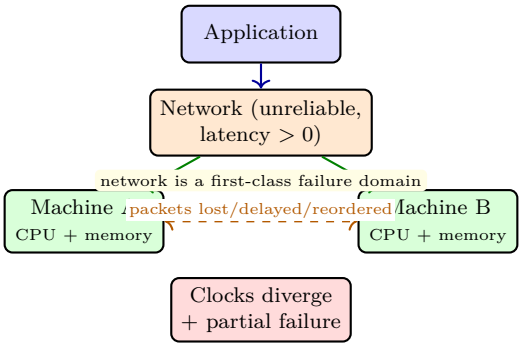

Figure 1.1: From one machine to two: the network and independent clocks/failures become part of the model.

1.2 System and Failure Models

A *system model* fixes what we assume about processes, links and time; a *failure model* fixes how things can break. Both bound what algorithms can guarantee.

Timing models. *Synchronous*: there are known bounds on processing time and message delay and clocks have bounded drift—rare in the Internet but useful for analysis. *Asynchronous*: no bounds at all. Because FLP (Ch. 4) shows consensus is impossible in async with one crash, real systems assume *partial synchrony*: bounds exist but are unknown or hold only eventually (Dwork–Lynch–Stockmeyer). Practical timeouts live here.

Failure models (ordered by severity).

Crash-stop (fail-stop)
A process halts and never recovers; others can detect it (eventually). Cleanest to reason about.

Crash-recovery
A process may crash and later restart with stable storage intact (write-ahead log survives). Most databases assume this.

Omission
Messages may be lost (send-omission or receive-omission) but processes do not lie.

Byzantine (arbitrary)
A faulty process may do anything—send conflicting messages, collude, act maliciously. Needed for open or adversarial settings.

Definition 1.1 (Fail-stop vs. fail-recovery vs. Byzantine)**.** A *fail-stop* process either follows its program or halts visibly. A *fail-recovery* process may halt and later resume from stable storage. A *Byzantine* process may deviate arbitrarily from its specification. Tolerating Byzantine faults with f faults needs $n \geq 3f + 1$ replicas (vs. $2f + 1$ for crash faults).

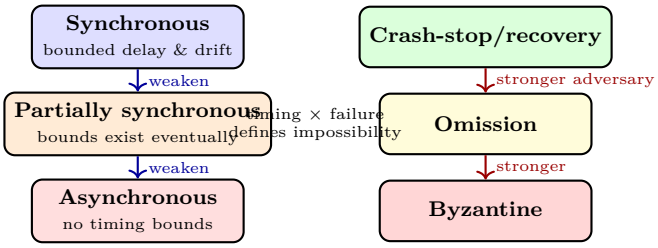

Figure 1.2: Two orthogonal model dimensions: timing assumptions (vertical left) and failure severity (vertical right).

Timing and failure together determine what is implementable. Example: perfect failure detection is possible in synchronous crash-stop but impossible in asynchronous—any timeout may confuse slowness with failure.

1.3 The CAP Theorem

The CAP theorem (Brewer, 2000; Gilbert–Lynch, 2002) says a read/write register replicated over a network that can partition cannot be both *linearizable* (consistent) and *available* on every request during the partition. Formally, of Consistency, Availability and Partition-tolerance, you can have at most two—and since partitions happen, the real choice is CP vs. AP.

Definition 1.2 (CAP properties for a read/write register)**.** *Consistency* means linearizability: every read returns the most recent write in a global real-time order. *Availability* means every non-faulty node returns a non-error response to every request. *Partition tolerance* means the system continues despite arbitrary message loss between groups of nodes.

Proof sketch (partition argument). Partition nodes into G_1 and G_2 with all cross messages lost. Client c_1 writes v to G_1; client c_2 concurrently reads from G_2. If the system is available, G_2 must answer. If it is

consistent, it must return v, but it has no way to learn v across the partition. So one property must be sacrificed. The system must either block/error (sacrifice availability) or return stale data (sacrifice consistency). □

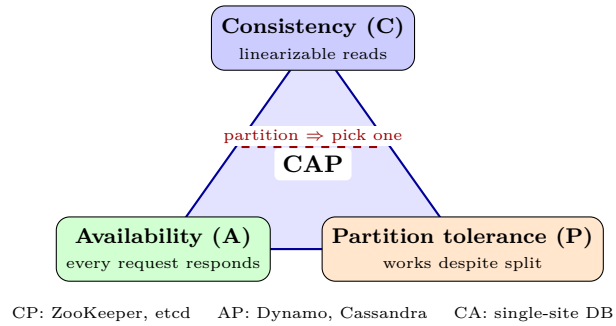

Figure 1.3: CAP triangle: under a partition you must choose C or A. Real systems also tune latency vs. consistency within that choice.

CAP is not the whole story. CAP talks about a single register under partition. Real trade-offs are finer: latency vs. consistency (PACELC), and many systems offer tunable consistency (read quorum R + write quorum W vs. N). Saying "we are AP" without stating which operations are available and with what staleness bound says little. Nevertheless CAP is the right first lens: it forces you to decide what happens when the network splits.

Example 1.1 (Classifying systems)**.** DNS is AP: it stays available under partition but may serve stale records (TTL-based). ZooKeeper is CP: it refuses writes on the minority side to keep linearizability. Dynamo-style stores let you choose $R + W > N$ for strong consistency or $R + W \leq N$ for availability and lower latency.

Remark 1.1 (Availability vs. fault tolerance). CAP's "availability" means every request to a non-failed node succeeds, which is stronger than "the service as a whole stays up somewhere." Most "highly available" systems are CP per CAP but highly available in the colloquial sense via failover and retries.

1.4 Abstractions: State Machine Replication

The central abstraction for fault tolerance is *state machine replication* (SMR): make n replicas behave like one correct state machine by feeding them the same commands in the same order. If replicas start in the same state and apply deterministic commands in identical order, they stay identical—so the service survives f faulty replicas.

SMR needs two ingredients: (1) a *total-order broadcast* (all correct replicas deliver messages in the same order) and (2) deterministic execution. Consensus (Ch. 4) implements total order; replication protocols (Ch. 5) implement it efficiently.

A tiny simulation makes the idea concrete.

```
# SMR intuition: deterministic replicas + total order => identical state
class Replica:
    def __init__(self, rid):
        self.id = rid
        self.state = {}          # key-value state machine
        self.log = []            # ordered command log
```

```
    def apply(self, cmd):
        # deterministic: same cmd on same state => same new state
        op, k, v = cmd
        if op == "put":
            self.state[k] = v
        elif op == "delete":
            self.state.pop(k, None)
        self.log.append(cmd)

# total-order broadcast delivers same sequence to every replica
commands = [("put","x",10), ("put","y",20), ("put","x",15)]
replicas = [Replica(i) for i in range(3)]
for cmd in commands:            # same order to all replicas
    for r in replicas:
        r.apply(cmd)

assert all(r.state == replicas[0].state for r in replicas)
print("all replicas converge:", replicas[0].state)
# {'x': 15, 'y': 20}  -- order matters: x=15 not 10
```

Change the delivery order on one replica and the states diverge—that is the failure SMR prevents. The rest of this book is largely about how to achieve total order under failures, asynchrony and scale.

Example 1.2 (Replicated counter with lost update)**.** Two clients concurrently increment a counter at replicas R_1, R_2 without total order: R_1 sees *inc* then *inc* (value 2), R_2 sees the two *inc* in opposite order but still 2—commutative here. Replace *inc* with $x := x + 1$ plus a conditional *if* $x > 5$ *then alert*: order now matters for whether the alert fires. SMR ensures every replica evaluates the condition in the same order, so alerts are consistent.

Remark 1.2 (Why SMR needs consensus). Total-order broadcast is equivalent to consensus: deciding the k-th log entry is a consensus instance. Hence FLP (Ch. 4) applies—SMR is impossible in pure async with one crash, which is why practical SMR (Raft, Paxos, Viewstamped Replication) assumes partial synchrony and elects a leader.

1.5 Exercises

E1.1 **Fallacies in the wild.** For each of fallacies 1, 2 and 4, describe a concrete user-visible failure in a mobile payment app and a design mitigation (timeout, retry budget, mTLS, etc.). Why does retrying blindly on an idempotent-unsafe operation make fallacy 1 worse?

E1.2 **Timing models and FLP.** Explain why a 100 ms client timeout cannot implement perfect failure detection in an asynchronous network. How does partial synchrony rescue the argument? State the FLP impossibility in one sentence.

E1.3 **CAP proof reconstruction.** Reconstruct the Gilbert–Lynch partition argument for a single register. Extend it to argue that no algorithm can guarantee both linearizability and availability for *every* operation during a partition, even with unbounded retries.

E1.4 **Failure model hierarchy.** Order crash-stop, crash-recovery, omission and Byzantine from weakest to

strongest adversary. For each step, give an example fault the stronger model captures but the weaker does not. Why does Byzantine tolerance need $n \geq 3f + 1$?

E1.5 **Classify and tune.** Classify DNS, etcd/ZooKeeper and Dynamo/Cassandra on the CAP triangle. For a Dynamo-style store with $N = 3$, enumerate (R, W) pairs giving strong consistency vs. eventual consistency and compute the read/write availabilities under one replica down.

Chapter Notes

Lamport's definition and Deutsch's fallacies are collected in Rotem-Gal-Oz, *Fallacies of Distributed Computing* (2006). System models follow Cachin–Guerraoui–Rodrigues, *Introduction to Reliable and Secure Distributed Programming* (2011, Ch. 2) and Coulouris et al., *Distributed Systems* (5th ed.). CAP: Brewer's PODC keynote (2000) and Gilbert–Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services" (SIGACT News, 2002). PACELC: Abadi (2012). SMR: Schneider, "Implementing fault-tolerant services using the state machine approach" (ACM Comp. Surv., 1990).

Chapter 2

Time, Clocks and Consistency

> *"Time is what we read from a clock."* — Einstein, operationally. In a distributed system there is no single clock to read, so we must invent notions of "before" that we can actually measure.

From zero: clocks and causality from scratch. We start with why wall clocks fail (drift, NTP limits), define the happens-before relation, then build logical and vector clocks that capture causality without physical time. We close by mapping the zoo of consistency models onto what applications actually need.

Learning Outcomes

After this chapter you will be able to:

(L1) explain clock drift, skew and NTP/PTP bounds (TrueTime);

(L2) define Lamport's happens-before and derive logical clock rules;

(L3) maintain and compare vector clocks to detect causality vs. concurrency;

(L4) order consistency models from strong to weak (linearizable → eventual);

(L5) choose a consistency level for a given application invariant.

2.1 Physical Time Is Unreliable

Two quartz oscillators drift apart by 10–100 parts per million; after a day that is seconds of skew. NTP synchronises to tens of milliseconds over the WAN (sub-millisecond on LAN); PTP with hardware timestamps reaches microseconds; Google's TrueTime (Spanner) exposes an *interval* $[earliest, latest]$ rather than a point, with typical uncertainty $\epsilon \approx 2$–$7\,\text{ms}$. No protocol can make clocks agree exactly in the presence of uncertain delay—it can only bound the error.

That bound matters. If node A timestamps an event at t_A and B at t_B, $t_A < t_B$ does *not* mean A happened first when $|t_A - t_B| < \epsilon$. Systems that order by wall-clock alone (e.g., "last writer wins" with client clocks)

silently lose updates when clocks skew.

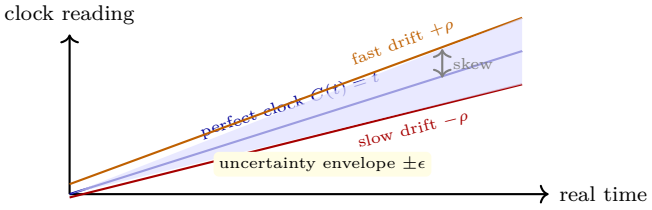

Figure 2.1: Physical clocks diverge within a drift envelope. NTP/PTP/TrueTime bound the skew but never eliminate it.

TrueTime's insight: instead of pretending *now* is a point, return $[now - \epsilon, now + \epsilon]$ and make callers wait out the uncertainty before committing. Spanner's commit-wait (Ch. 7) is exactly that: delay commit until $TT.now().latest$ passes the chosen timestamp, so timestamp order respects real-time order.

2.2 Lamport Clocks: Capturing Happens-Before

Lamport (1978) asked what "before" can mean without physical time. Define the *happens-before* relation $\rightarrow$ as the smallest transitive relation such that (i) if a and b are events on the same process and a precedes b, then $a \rightarrow b$, and (ii) if a is the send of a message and b is its receive, then $a \rightarrow b$. Two events are *concurrent* ($a \parallel b$) if neither $a \rightarrow b$ nor $b \rightarrow a$.

A *Lamport logical clock* C is a counter per process:

$$\begin{aligned} \text{local event / send:} \quad & C := C + 1 \\ \text{receive of } m \text{ with } C_m: \quad & C := \max(C, C_m) + 1; \ \text{deliver } m \end{aligned} \tag{2.1}$$

with C_m piggybacked on m.

Theorem 2.1 (Clock condition)**.** If $a \rightarrow b$ then $C(a) < C(b)$. The converse does not hold.

So equal or smaller clocks do not prove causality—they only rule it out in one direction. That gap motivates vector clocks.

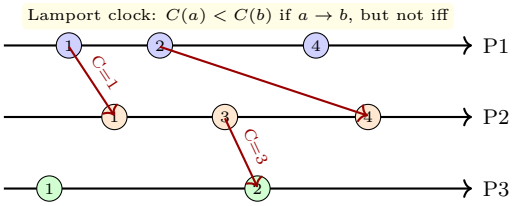

Figure 2.2: Lamport clocks along three processes. Messages carry the sender's clock; receivers advance past it. Only the clock condition (one direction) holds.

Total order from Lamport clocks is obtained by breaking ties with process IDs: $(C(a), pid(a)) < (C(b), pid(b))$. That order respects $\rightarrow$ and is useful for mutual exclusion (Lamport's bakery), but it *imposes* order on concurrent events arbitrarily—sometimes you need to know they were concurrent.

2.3 Vector Clocks: Detecting Causality

A *vector clock* VC is an array of n counters, one per process. On process i:

$$\begin{aligned} \text{local/send:} \quad & VC[i] := VC[i] + 1 \\ \text{receive of } m \text{ with } VC_m: \quad & VC[j] := \max(VC[j], VC_m[j])\ \forall j;\ VC[i]{+}{+} \end{aligned} \tag{2.2}$$

Comparison is componentwise:

$$VC(a) < VC(b) \iff \forall k\, VC(a)[k] \le VC(b)[k] \ \wedge\ \exists k\, VC(a)[k] < VC(b)[k]. \tag{2.3}$$

Theorem 2.2 (Vector clock characterisation)**.** $VC(a) < VC(b) \iff a \to b$. $VC(a) \parallel VC(b)$ (incomparable) $\iff a \parallel b$.

Thus vector clocks *exactly* capture causality, at cost $O(n)$ per message. Practical systems use optimisations: dotted version vectors, version vectors per key, or causal histories pruned by stability.

```python
# Vector clocks: exact causality tracking
class VectorClock:
    def __init__(self, n, pid):
        self.n, self.pid = n, pid
        self.v = [0]*n
    def tick(self):
        self.v[self.pid] += 1
        return list(self.v)
    def send(self):
        self.v[self.pid] += 1
        return list(self.v)          # piggyback on message
    def receive(self, other):
        for i in range(self.n):
            self.v[i] = max(self.v[i], other[i])
        self.v[self.pid] += 1

# Example: 3 processes, concurrent updates detected
a = VectorClock(3, 0); b = VectorClock(3, 1)
a.tick()            # a: [1,0,0]
b.tick()            # b: [0,1,0]  -- incomparable => concurrent
print(a.v, b.v, "concurrent:", not (a.v < b.v or b.v < a.v))

# After b receives from a, b dominates a => causal
msg = a.send()      # a: [2,0,0]
b.receive(msg)      # b: [2,2,0]
print("after receive b=", b.v, " a->b ?", all(x<=y for x,y in zip([2,0,0], b.v)))
```

2.4 Consistency Models

Consistency defines what values a read may return given concurrent writes. Stronger models are easier to program but cost latency and availability.

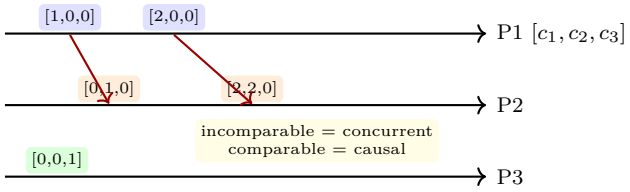

Figure 2.3: Vector clocks on three processes. $[1,0,0] \parallel [0,1,0]$ (concurrent) but $[1,0,0] < [2,2,0]$ (causal).

Linearizability (strong)

Every operation appears to take effect atomically at some point between invocation and response, respecting real-time order. What a single register "should" do. Cost: needs consensus on the critical path.

Sequential consistency

All processes see the same order of operations, and each process's own operations appear in program order, but real-time order across processes need not hold. Cheaper than linearizable, still strong.

Causal consistency

Only causally related operations are ordered; concurrent writes may be seen in different orders. Captured exactly by vector clocks. Available under partition.

Eventual consistency

If no new writes, eventually all replicas converge. No ordering guarantee before that. Dynamo/Cassandra default; needs conflict resolution.

Theorem 2.3 (Hierarchy)**.** Linearizability ⇒ sequential ⇒ causal ⇒ eventual. No implication reverses.

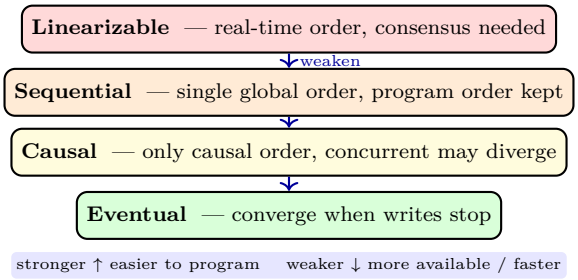

Figure 2.4: Consistency spectrum. Moving down trades programmability for latency, throughput and partition availability.

Choosing a level is an application decision. A bank ledger needs linearizable transfer; a social-media like counter can be eventually consistent; a collaborative editor needs causal (so replies follow their parent) but not linearizable.

```python
# Simulating causal delivery with vector clocks (buffer until causally ready)
from collections import defaultdict, deque

def causally_ready(msg_vc, delivered_vc, sender):
    # ready iff msg is next from sender and not ahead on any other entry
    if msg_vc[sender] != delivered_vc[sender] + 1:
        return False
    return all(msg_vc[k] <= delivered_vc[k] for k in range(len(msg_vc)) if k != sender)

# Three messages: m1 from P0, m2 from P1 concurrent, m3 from P1 after seeing m1
delivered = [0,0]
```

```
buf = deque([([0,1],1), ([1,0],0), ([1,1],1)])  # out-of-order arrival
while buf:
    vc, sender = buf[0]
    if causally_ready(vc, delivered, sender):
        delivered = [max(delivered[i], vc[i]) for i in range(2)]
        print(f"deliver from P{sender} vc={vc} -> delivered={delivered}")
        buf.popleft()
    else:
        buf.rotate(-1)  # try next
        # in real system: wait for missing predecessor
        if all(not causally_ready(vc,s,delivered) for vc,s in buf):
            break
```

2.5 Exercises

E2.1 **NTP bounds.** Two machines sync via NTP with round-trip 20 ms and server processing 2 ms. Derive the bound on clock offset. If TrueTime reports $\epsilon = 5$ ms, how long must Spanner's commit-wait hold? Why is an interval better than a point timestamp?

E2.2 **Lamport trace.** Three processes exchange messages as in Fig. 2.2. Compute Lamport timestamps for every event from scratch. Identify a pair a, b with $C(a) < C(b)$ but $a \not\to b$ and explain why.

E2.3 **Vector clock application.** Processes P_1 and P_2 concurrently write key x with vector clocks $[1, 0]$ and $[0, 1]$. A replica holds both versions. Show the clocks are incomparable. How does Dynamo-style reconciliation expose this to the application? Propose a merge function for a shopping cart.

E2.4 **Consistency hierarchy.** Prove sequential consistency does not imply linearizability with a concrete 2-process execution. Then show causal consistency does not imply sequential with a 3-process example where concurrent writes are seen in different orders.

E2.5 **Choosing consistency.** For each workload—(a) seat reservation, (b) page-view counter, (c) chat with replies—choose the weakest consistency that preserves correctness. Justify and name a system or protocol that provides it.

Chapter Notes

Lamport, "Time, clocks, and the ordering of events in a distributed system" (CACM, 1978) introduces happens-before and logical clocks. Vector clocks: Fidge (1988) and Mattern (1989) independently. Physical time and TrueTime: Corbett et al., "Spanner" (OSDI, 2012). Consistency hierarchy: Lamport (1979) for sequential; Herlihy–Wing (1990) for linearizability; causal memory: Ahamad et al. (1995). Comprehensive treatment: Kshemkalyani–Singhal, *Distributed Computing* (2nd ed., Ch. 6–7) and van Steen–Tanenbaum (3rd ed., Ch. 6).

Chapter 3

Remote Calls, Serialization and Messaging

> *"Remote procedure call is the assembly language of distributed computing."* — Jim Waldo. It looks like a function call, but the network is between caller and callee, and that changes everything.

From zero: from local calls to network calls. We start with what a local call guarantees, add the network, and see why transparency is leaky. We then cover how data is encoded for the wire, how RPC is implemented, and when to use messaging instead. Every idea is grounded in runnable Python.

Learning Outcomes

After this chapter you will be able to:

(L1) explain why RPC cannot be fully transparent (partial failure, latency, at-most/at-least/exactly-once);

(L2) compare serialization formats (JSON, Protocol Buffers, Avro) on schema evolution and efficiency;

(L3) describe gRPC/Thrift stub generation and streaming modes;

(L4) contrast RPC (request–reply) with message queues and pub/sub on coupling and durability;

(L5) implement idempotency keys and retry budgets to achieve effective exactly-once.

3.1 From Local Calls to Remote Calls

A local call is *total*: it either returns a value or raises an exception, and failure means the whole process failed. A remote call can fail *partially*: the request was sent but the reply was lost, so the caller does not know whether the callee executed. This is the fundamental transparency break.

Waldo et al. (1994) argued RPC can never be fully transparent because objects differ in failure, latency, memory access and concurrency. Hiding the network behind the same syntax invites fallacy 1 and 2 (Ch. 1) into application code.

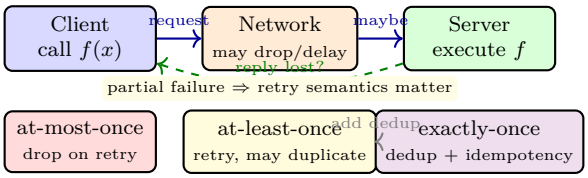

Figure 3.1: RPC partial failure: did the server run f? The answer determines at-most vs. at-least vs. effective exactly-once.

Retry semantics.

At-most-once
Send once; on timeout return error. Safe against duplication but loses operations.

At-least-once
Retry until ack; may execute multiple times. Requires idempotent handler or deduplication.

Exactly-once (effective)
At-least-once + server-side dedup via idempotency key + idempotent side effects. True exactly-once over an unreliable network is impossible in general; this is the practical approximation.

Definition 3.1 (Idempotency)**.** An operation f is *idempotent* if $f(f(x)) = f(x)$: repeating it has the same effect as doing it once. Example: $x := 5$ is idempotent; $x{+}{+}$ is not. Non-idempotent operations need an idempotency key so the server can recognise retries.

3.2 Serialization and Interface Definition

Before bytes go on the wire they must be *serialized* (marshalled). Choices trade readability, size, speed and schema evolution.

Format	Human-readable	Size	Schema	Evolution
JSON	yes	large	none	ad-hoc
XML	yes	larger	XSD	verbose
Protocol Buffers	no	small	.proto	excellent (field tags)
Avro	no	small	JSON schema	excellent (reader/writer)
Thrift	no	small	.thrift	good

Table 3.1: Serialization trade-offs. Binary IDLs win on size and evolution; JSON wins on debugging.

Schema evolution rules. Adding a field must not break old readers (use optional with defaults); removing a field must reserve its tag; changing a type is often incompatible. Protobuf's field numbers and Avro's reader/writer schemas exist precisely to make evolution safe—JSON's flexibility is also its danger (silent missing-field bugs).

```
# Serialization comparison: JSON vs Protobuf-style varint (illustrative)
import json, struct

# JSON: readable but verbose, no schema enforcement
payload = {"user_id": 42, "action": "deposit", "amount": 100}
json_bytes = json.dumps(payload).encode()
```

```
print("JSON bytes:", len(json_bytes), json_bytes[:40])

# Simulated protobuf varint for an integer field (LEB128)
def varint_encode(n):
    out = bytearray()
    while n >= 0x80:
        out.append((n & 0x7F) | 0x80)
        n >>= 7
    out.append(n)
    return bytes(out)

for v in [1, 127, 128, 300, 10_000]:
    print(f"{v:6d} -> varint {varint_encode(v).hex()} ({len(varint_encode(v))}B)")

# Idempotency key pattern: server deduplicates by key
store = {}
def handle_request(key, op):
    if key in store:          # duplicate retry
        return store[key]
    result = op()             # execute once
    store[key] = result
    return result
```

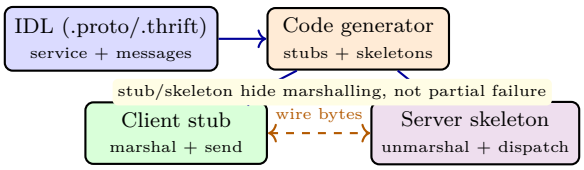

Figure 3.2: IDL-driven RPC: the interface definition generates client stubs and server skeletons; the wire format is fixed by the IDL compiler.

3.3 gRPC, Thrift and Streaming

Modern RPC frameworks share a pattern: define the service in an IDL, generate code, run over HTTP/2 or a custom transport.

gRPC (Protobuf + HTTP/2). Four modes: unary (one request, one response), server-streaming, client-streaming, bidirectional streaming. HTTP/2 multiplexing, flow control and header compression come for free. Interceptors provide auth, tracing and retry budgets.

Thrift (Apache). Similar IDL with multiple transports (framed, header) and protocols (binary, compact, JSON). Widely used at Meta.

Streaming matters because *not every interaction is request–reply.* A log tail or price feed is naturally a stream; polling it via unary RPC wastes latency and load.

```
# Retry with idempotency key and exponential backoff + jitter (client side)
import random, time, uuid
```

```python

def rpc_with_retry(stub_call, max_retries=4, base_ms=50):
    key = str(uuid.uuid4())          # idempotency key reused across retries
    for attempt in range(max_retries + 1):
        try:
            return stub_call(idempotency_key=key)
        except TimeoutError:
            if attempt == max_retries:
                raise
            # exponential backoff with jitter avoids thundering herd
            sleep_ms = base_ms * (2 ** attempt) + random.randint(0, 30)
            time.sleep(sleep_ms / 1000.0)
    # Server: if key seen before, return cached response (see earlier handle_request)

# Example: safe retry of a payment (non-idempotent without key)
def make_payment(amount):
    def call(key=None):
        # stub would send key in header; server deduplicates
        return f"charged {amount} (key={key[:8]}...)"
    return rpc_with_retry(lambda idempotency_key: call(idempotency_key))

print(make_payment(50))
```

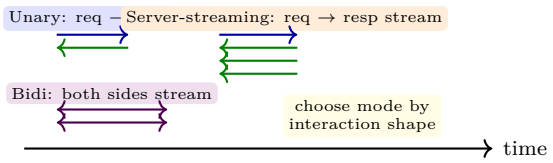

Figure 3.3: gRPC interaction modes. Streaming avoids polling when data flows continuously.

3.4 Messaging: Queues and Pub/Sub

RPC couples caller and callee in time (both must be up) and space (caller knows callee). Messaging decouples them.

Message queue (point-to-point)
Producer sends to a queue; one consumer receives. Durable queues (RabbitMQ, SQS, Kafka with consumer groups) survive consumer failure and smooth bursts. Semantics: at-least-once delivery, ordering per partition.

Publish/subscribe
Publisher sends to a topic; many subscribers receive. Decouples publishers from subscriber count (Kafka, SNS, Redis Pub/Sub). Ordering is per topic-partition, not global.

Log (Kafka-style)
An append-only partitioned log that is both queue and pub/sub: consumers track offsets, replay is possible, retention is time/size-based. The log is the database.

When to use which: RPC for request–reply with low latency and strong typing; queues for work distribution and back-pressure; pub/sub for event broadcast and fan-out; logs when you need replay and audit. Many

architectures mix them: an API gateway speaks gRPC to clients and publishes domain events to Kafka for downstream consumers.

Example 3.1 (Ordering pitfall)**.** Two producers send A then B to different Kafka partitions. A consumer reading both partitions may see B before A. If order matters, the producer must key both messages to the same partition (same key $\rightarrow$ same partition $\rightarrow$ total order per key).

3.5 Exercises

E3.1 **Partial failure analysis.** A client sends RPC *pay*($100) and times out. Enumerate the four possible server states (never received, executed but reply lost, executed and reply in flight, failed before execution). For each, state whether retrying is safe without an idempotency key.

E3.2 **Schema evolution.** A Protobuf message has fields 1:*user_id*, 2:*email*. You need to rename *email* to *contact_email* and add *phone*. Show the .proto diff that is wire-compatible with old binaries. What breaks if you reuse field number 2 for *phone*?

E3.3 **Retry budgets.** A service at 1000 RPS has a 1% timeout rate. With blind retry (one retry per timeout), compute the extra load during a 10% timeout spike. Propose a retry budget (e.g., 20% of requests) and explain how it prevents retry storms.

E3.4 **RPC vs. messaging.** Design the interaction for an image-upload service that must (a) return upload success quickly and (b) generate thumbnails asynchronously. Justify using RPC for (a) and a queue/log for (b). Where does ordering matter and how would you guarantee it?

E3.5 **Exactly-once simulation.** Extend the idempotency-key handler in the chapter's Python to persist the dedup table to disk (crash-recovery model). Show a trace where the server crashes after executing but before acking, restarts, and correctly deduplicates the client's retry.

Chapter Notes

Waldo et al., "A note on distributed computing" (1994) is the classic transparency critique. Birrell–Nelson (1984) introduced RPC. gRPC: `grpc.io`; Thrift: Apache Thrift whitepaper (2007). Serialization evolution: Kleppmann, *Designing Data-Intensive Applications*, Ch. 4 (2017) is the best single reference. Messaging semantics and Kafka logs: Kleppmann Ch. 11 and Kreps, "The Log" (2013).

Chapter 4

Consensus: Paxos, Raft and Byzantine Agreement

> *"Consensus is the distributed equivalent of agreeing on what time it is when no one has a correct watch."* — Barbara Liskov. Without agreement on order, replicas diverge; with it, they can act as one.

From zero: why agreement is hard. We build from a simple majority vote, show why naive voting fails under asynchrony and crashes, analyse Paxos and its understandable successor Raft, then extend to Byzantine faults. Runnable Python illustrates the core logic.

Learning Outcomes

After this chapter you will be able to:

(L1) define consensus (agreement, validity, termination) and state FLP impossibility;

(L2) trace Paxos prepare/promise/propose/accept and explain why a majority quorum suffices;

(L3) describe Raft leader election, log replication and commitment rules;

(L4) compare Paxos and Raft on understandability, liveness and leader transfer;

(L5) explain PBFT phases and the $n \geq 3f + 1$ bound for Byzantine agreement.

4.1 The Consensus Problem and FLP

n processes each propose a value; they must decide on one value satisfying:

Agreement
No two correct processes decide differently.

Validity

If all propose v then v is decided (some forms: decided value was proposed by someone).

Termination

Every correct process eventually decides.

This looks easy—take a majority vote. But messages can be delayed and processes can crash. The celebrated FLP result (Fischer–Lynch–Paterson, 1985) proves:

Theorem 4.1 (FLP impossibility)**.** No deterministic algorithm solves consensus in an asynchronous system with even one crash failure.

Consequence: every practical consensus algorithm either uses randomness or timing assumptions (timeouts, partial synchrony) to circumvent FLP. Paxos and Raft assume partial synchrony: the network is async but eventually timely enough for a leader to make progress.

4.2 Paxos: Single-Decree and Multi-Paxos

Paxos (Lamport, 1989/1998) has three roles: proposers, acceptors and learners. A value is *chosen* when a majority of acceptors accept it. Two phases:

Phase 1 (Prepare). Proposer picks ballot b higher than any seen and sends $\langle prepare, b\rangle$ to a majority. Each acceptor that has not promised a higher ballot replies with $\langle promise, b, (b_{max}, v_{max})\rangle$ (its highest accepted ballot/value, if any) and promises to ignore lower ballots.

Phase 2 (Accept). Proposer collects promises from a majority. If any acceptor reported a value, the proposer must propose the value with the highest b_{max} among them; otherwise it may propose its own. It sends $\langle accept, b, v\rangle$. Acceptors accept if they have not promised higher. When a majority accepts, v is chosen.

Safety hinges on quorum intersection: any two majorities overlap, so if v is chosen at ballot b, every higher ballot's Phase 1 will see v and be forced to propose it.

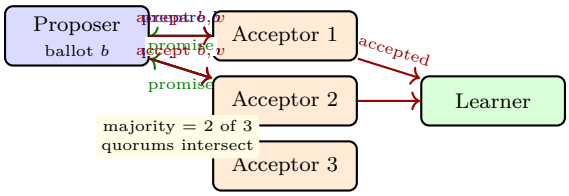

Figure 4.1: Paxos Phase 1 (prepare/promise) then Phase 2 (accept/accepted). A majority (2 of 3) suffices to choose a value.

Multi-Paxos runs Paxos for each log position but keeps the same leader across positions, so Phase 1 is done once per leader term and steady-state replication is just Phase 2—essentially Raft's happy path before Raft made it explicit.

4.3 Raft: Understandable Consensus

Raft (Ongaro–Ousterhout, 2014) decomposes consensus into three subproblems: leader election, log replication and safety. The design goal is understandability without sacrificing correctness.

Leader election. Time is divided into *terms* (monotonic integers). Each term has at most one leader. Followers use randomised election timeouts (e.g., 150–300 ms). On timeout, a follower becomes candidate, increments its term, votes for itself and requests votes. A candidate that gets a majority becomes leader. The randomisation reduces split votes.

Log replication. Clients send commands to the leader, which appends them to its log and replicates via AppendEntries RPCs. An entry is *committed* once a majority has it; then it is applied to the state machine. Followers overwrite conflicting entries to match the leader—Raft never reorders committed entries.

Safety: election restriction. A candidate wins only if its log is at least as up-to-date as any voter (compare last term, then length). This guarantees a new leader has every committed entry—no committed entry is ever lost.

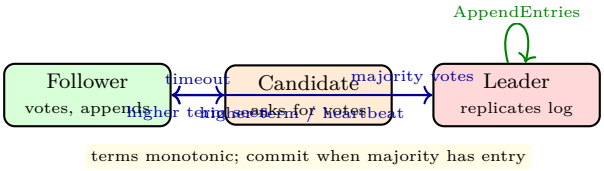

Figure 4.2: Raft state machine: follower → candidate on timeout, candidate → leader on majority, any state → follower on higher term. Leader replicates log.

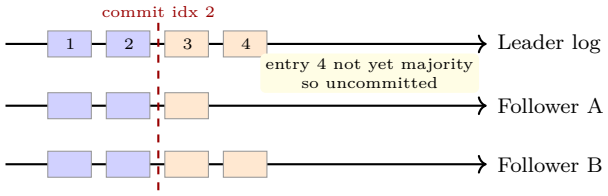

Figure 4.3: Raft log replication: leader appends entries and replicates; entry is committed when a majority stores it (index 2). Uncommitted entries may be overwritten.

```python
# Raft leader: replicate and commit when majority ack (simplified)
class RaftLeader:
    def __init__(self, n):
        self.n, self.log = n, []          # log is list of (term, cmd)
        self.term = 1
        self.commit_idx = -1
        self.match_idx = [-1]*n           # highest replicated idx per follower
    def append(self, cmd):
        self.log.append((self.term, cmd))
        return len(self.log)-1
    def on_ack(self, follower, idx):
        self.match_idx[follower] = max(self.match_idx[follower], idx)
        # commit if majority has idx and term matches current term
        sorted_match = sorted(self.match_idx + [len(self.log)-1])
```

```
        majority_idx = sorted_match[len(sorted_match)//2]
        if majority_idx > self.commit_idx and self.log[majority_idx][0]==self.term:
            self.commit_idx = majority_idx
            print(f"committed up to index {self.commit_idx}: {self.log[:self.commit_idx
                +1]}")

leader = RaftLeader(5)
leader.append("x:=1"); leader.append("y:=2")
leader.on_ack(1, 1); leader.on_ack(2, 1)  # 3 of 5 have idx 1 -> commit 1
leader.on_ack(3, 0)                        # not enough for idx beyond
```

4.4 Byzantine Fault Tolerance

When faulty nodes may lie, crash-fault consensus fails. Byzantine agreement needs $n \geq 3f+1$ (Pease–Shostak–Lamport, 1980) and typically $O(n^2)$ messages. PBFT (Castro–Liskov, 1999) is the canonical protocol.

PBFT has three phases per request: *pre-prepare* (primary broadcasts ordering), *prepare* (replicas broadcast that they accept the ordering, $2f$ matching prepares form a prepared certificate), and *commit* ($2f+1$ matching commits guarantee linearizability). A view change replaces a faulty primary. Modern BFT (HotStuff, Tendermint) linearises communication via the leader to reduce messages.

```
# Byzantine quorum intuition: why 3f+1? Two quorums of size 2f+1 overlap in f+1 nodes,
# so even if f of them are Byzantine, at least one honest node is in the overlap.
def quorum_overlap(n, f):
    q = 2*f + 1
    overlap = 2*q - n
    honest_in_overlap = overlap - f      # worst case: f Byzantine in overlap
    return q, overlap, honest_in_overlap

for f in [1,2,3]:
    n = 3*f + 1
    q, ov, hon = quorum_overlap(n, f)
    print(f"f={f} n={n} quorum={q} overlap={ov} honest_in_overlap={hon} (need >=1)")

# With n=3f, overlap=f, honest_in_overlap could be 0 -> safety violated
print("n=3f fails:", quorum_overlap(6, 2))
```

Remark 4.1 (Paxos vs. Raft in practice). Paxos is more flexible (any proposer, no distinguished leader needed for correctness) but harder to implement correctly; Raft pins leadership to simplify reasoning and adds the election restriction to guarantee leader completeness. Most new systems choose Raft for operability; Paxos variants (EPaxos, WPaxos) win when leader bottlenecks dominate.

Example 4.1 (Raft log matching)**.** Leader with log $[a_1, b_1]$ and follower with $[a_1]$: the AppendEntries consistency check (prevLogIndex, prevLogTerm) fails for any entry that would diverge, so the leader decrements `nextIndex` and retries with older entries until the logs align, then overwrites the tail. This is how Raft repairs a follower that missed writes while partitioned.

Remark 4.2 (Commit latency). Consensus latency is at least one round-trip to a majority. Raft batches AppendEntries to amortise this, and followers can serve linearizable reads via a heartbeat round-trip or lease read;

otherwise stale reads are faster but may violate linearizability (the read-index optimisation).

4.5 Exercises

E4.1 **FLP and timeouts.** State FLP precisely and explain why adding a perfect failure detector would circumvent it. Why is implementing a perfect detector equivalent to solving consensus in async?

E4.2 **Paxos trace.** With 3 acceptors, proposer P_1 with ballot 1 proposes $v = a$ and reaches only acceptor A_1; then P_2 with ballot 2 runs Phase 1 contacting A_2, A_3. What value must P_2 propose? Show the quorum-intersection argument that forces this choice.

E4.3 **Raft election.** In a 5-node Raft cluster, follower F has log $[(1, x), (1, y)]$ (last term 1). Candidate C has $[(1, x)]$ and term 2. Can C win election if F is among the voters? State Raft's election restriction and apply it.

E4.4 **Commit rule.** Explain why Raft only commits entries from the current term via majority. Give a scenario where committing an old-term entry by counting replicas would violate safety (the figure-8 example in the Raft paper).

E4.5 **PBFT quorum.** Prove $n \geq 3f + 1$ is necessary for Byzantine agreement with oral messages (sketch the partitioning argument). For $n = 7$, what is the largest f tolerated and what quorum size does PBFT use for prepare and commit?

Chapter Notes

Lamport, "The part-time parliament" (1998) for Paxos; Paxos Made Simple (2001) is the readable version. Ongaro–Ousterhout, "In search of an understandable consensus algorithm" (USENIX ATC, 2014) for Raft. FLP: Fischer–Lynch–Paterson (JACM, 1985). Byzantine generals: Lamport–Shostak–Pease (1982). PBFT: Castro–Liskov (OSDI, 1999). HotStuff: Yin et al. (PODC, 2019). For a unified view see Cachin et al., Ch. 5 and Kleppmann Ch. 9.

Chapter 5

Replication: Primary-Backup, Quorums and Chains

"Replication is the art of keeping copies close enough to be useful and far enough to survive." We want every read to see the right value, even when some copies are slow, stale or gone.

From zero: why copy data and how to keep copies consistent. We start with a single copy, add a second for availability, and discover the spectrum from eager to lazy replication. We then compare three families—primary-backup, quorum and chain—on consistency, latency and failure handling.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish synchronous vs. asynchronous replication and state the data-loss window;

(L2) describe primary-backup with log shipping and failover, including fencing;

(L3) apply quorum replication ($R + W > N$) and tune R, W for read/write trade-offs;

(L4) explain chain replication and its linearizable, high-throughput pipeline;

(L5) reason about replica consistency (read repair, anti-entropy, hinted handoff).

5.1 Why Replicate?

Replication serves three goals: *availability* (survive f failures with $f+1$ copies, or $2f+1$ for strong consistency), *durability* (geographic copies survive site failure), and *performance* (nearby copy reduces latency, many copies increase read throughput). The price is consistency: writes must reach multiple copies, and the network between them is the bottleneck.

Two extremes frame the space. *Eager (synchronous)* replication waits for replicas before acking the write—strong consistency, high latency, lower availability during partitions. *Lazy (asynchronous)* replication acks after one copy and propagates in the background—low latency, higher availability, but reads may be stale and un-replicated writes can be lost on failover.

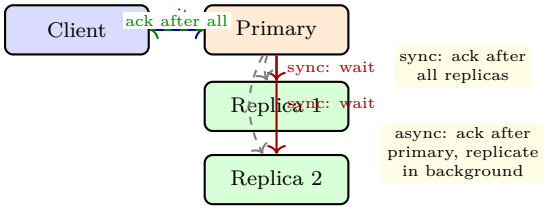

Figure 5.1: Synchronous vs. asynchronous replication. The dashed background arrows show async propagation that does not block the client ack.

5.2 Primary-Backup (Single-Leader)

One replica is the *primary* (leader); all writes go through it. The primary orders writes, appends to its log, ships the log to backups, and acks after the chosen durability level (1 ack for async, majority for semi-sync, all for sync).

Failover. When the primary fails, a backup is promoted. The hard problems are (i) *detecting* failure without false positives (lease or heartbeat + timeout), (ii) *fencing* the old primary so it cannot keep serving (STONITH, epoch/fencing token), and (iii) *deciding* how much of the old primary's unreplicated tail to keep—discarding it loses acknowledged writes; keeping it needs consensus.

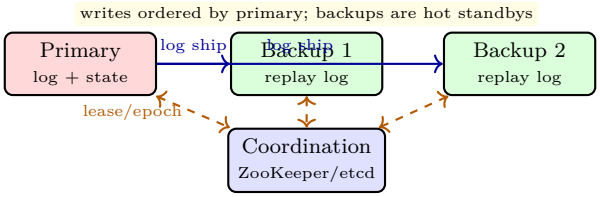

Figure 5.2: Primary-backup with log shipping. A coordination service holds the lease/epoch and fences the old primary on failover.

```python
# Primary-backup: log shipping with fencing token (epoch)
class Primary:
    def __init__(self, epoch=1):
        self.epoch, self.log = epoch, []
        self.backups = []
    def add_backup(self, b): self.backups.append(b)
    def write(self, cmd):
        # fencing token (epoch) travels with every RPC so old primary is rejected
        self.log.append((self.epoch, cmd))
        acks = 1  # primary itself
        for b in self.backups:
            if b.replicate(self.epoch, self.log[-1]):
                acks += 1
        return acks

```

```python
class Backup:
    def __init__(self): self.log, self.epoch = [], 0
    def replicate(self, epoch, entry):
        if epoch < self.epoch:   # fenced old primary
            return False
        self.epoch = epoch
        self.log.append(entry)
        return True

p, b1, b2 = Primary(epoch=3), Backup(), Backup()
p.add_backup(b1); p.add_backup(b2)
print("acks:", p.write("x:=10"))

# Failover: new primary with higher epoch fences old primary
p_new = Primary(epoch=4)
b1.epoch = 0  # simulate b1 accepting new primary
print("old primary after fencing:", p.write("y:=20"), " (b1 rejects if fenced)")
```

Without fencing, a network-partitioned old primary that still believes it is primary can serve stale writes—the classic *split-brain*. Fencing tokens (monotonic epoch written to a coordination service) make the old primary's writes rejectable.

5.3 Quorum Replication

In *quorum replication* (Gifford, 1979) there is no single primary. With N replicas, a write contacts W replicas and a read contacts R. If $R + W > N$, every read quorum overlaps every write quorum, so the read sees the latest write. Similarly $W + W > N$ ensures writes overlap (for compare-and-swap).

This gives tunable consistency:

$$R + W > N \;\wedge\; W > N/2 \quad\Longrightarrow\quad \text{strong (linearizable) with version numbers.} \tag{5.1}$$

If $R + W \le N$, reads may be stale (eventual). Dynamo and Cassandra expose R, W per operation so the same cluster can serve strong reads ($R = N, W = 1$ is also overlapping but slow) or fast eventual reads ($R = 1, W = 1$).

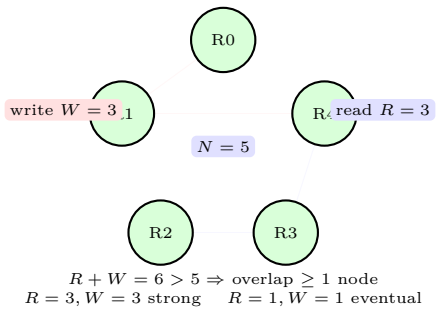

Figure 5.3: Quorum overlap: with $N = 5$, $R = 3, W = 3$ any read quorum meets any write quorum. Shrinking R or W trades consistency for latency.

Repair. Quorum reads may discover divergent versions. *Read repair* pushes the newest version to stale replicas; *anti-entropy* (Merkle trees, gossip) converges replicas in the background; *hinted handoff* temporarily stores a write for a down replica on another node and replays when it returns.

```python
# Quorum replication simulation: R+W > N guarantees overlap
N, R, W = 5, 3, 3
replicas = {i: (0, None) for i in range(N)}  # (version, value)

def quorum_write(value, version, w_nodes):
    for n in w_nodes: replicas[n] = (version, value)

def quorum_read(r_nodes):
    # return value with highest version among read quorum
    best = max((replicas[n] for n in r_nodes), key=lambda x: x[0])
    # read-repair: push best to stale nodes in quorum
    for n in r_nodes:
        if replicas[n][0] < best[0]:
            replicas[n] = best
    return best

quorum_write("hello", version=1, w_nodes=[0,1,2])
print("read from [2,3,4]:", quorum_read([2,3,4]))  # must see version 1 via node 2
print("read from [3,4,0]:", quorum_read([3,4,0]))  # also sees it; repair heals 3,4
print("replicas:", replicas)
```

5.4 Chain Replication

Chain replication (van Renesse–Schneider, 2004) arranges N replicas in a chain. Writes enter at the *head*, propagate through the chain, and are acked by the *tail*. Reads go to the tail for strong consistency (or to any node for eventual).

The chain gives linearizable order without a separate consensus round per write: the chain *is* the order. Throughput is high because nodes form a pipeline (head processes write $k+1$ while tail acks k). Failure handling needs a coordinator to splice out a failed node and re-chain.

Chain replication is the core of strongly consistent stores like CRAQ and of log systems where the chain maps to a replicated log.

Example 5.1 (Chain read optimisation)**.** CRAQ (Chain Replication with Apportioned Queries) lets any chain node serve a *dirty* read locally if it has seen the write, marking it dirty until the tail acknowledges. Clean reads go to the tail. This keeps strong consistency while spreading read load—useful when reads dominate writes by 10–100×.

Remark 5.1 (Choosing a replication family). Primary-backup is simplest when writes are naturally funnelled through one leader (single-key operations). Quorums shine for multi-key, leaderless workloads with tunable latency. Chains excel for log-structured, append-heavy stores where pipeline throughput matters. Many production stores combine them: e.g., a Paxos-replicated primary whose backups form a chain for log shipping.

5.5 Exercises

E5.1 **Sync vs. async window.** A primary acks after the primary only (async) and ships to 2 backups with 10 ms delay. If the primary crashes uniformly at random in a 1 s window, what is the expected number of lost acknowledged writes at 100 writes/s? How does semi-sync ($W = 2$) change the answer?

E5.2 **Fencing.** Without fencing, describe a split-brain scenario with a partitioned primary that keeps serving writes. Show how an epoch/fencing token stored in ZooKeeper prevents the old primary's writes from being accepted by backups after failover.

E5.3 **Quorum tuning.** With $N = 5$, enumerate (R, W) pairs for (a) strong consistency, (b) read-optimised strong, (c) write-optimised strong, (d) eventual. For each, state how many replica failures can be tolerated for reads and writes.

E5.4 **Chain vs. quorum.** Compare chain replication and quorum replication on (i) read latency, (ii) write throughput under pipeline, (iii) behaviour when the tail/head fails, (iv) load distribution. When would you prefer one over the other?

E5.5 **Anti-entropy.** Two replicas hold 1M keys each, differing in 10 keys. Explain why naive full comparison is wasteful and how Merkle trees let you find the differing keys in $O(\log n)$ exchanges. Sketch the tree walk.

Chapter Notes

Gifford (1979) introduced weighted voting/quorums. Primary-backup and viewstamped replication: Oki–Liskov (1988). Chain replication: van Renesse–Schneider (OSDI, 2004); CRAQ: Terra–Shah (2009). Quorum tuning and Dynamo: DeCandia et al., "Dynamo" (SOSP, 2007). Fencing and leases: Kleppmann, Ch. 8–9. Anti-entropy and Merkle trees: Demers et al. (1987) and Dynamo's use of them.

Chapter 6

Distributed Storage: Files, Blocks and Objects

"A file system is a database with a funny API; a distributed file system is a database that must also survive partitions." We layer that idea from files to blocks to objects.

From zero: what "storing a file" means when disks are remote. We start with a local file, make the disk remote, then replicate and stripe it. We contrast file, block and object abstractions and study HDFS and S3 as canonical designs.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish file, block and object storage on interface, mutability and scale;

(L2) explain HDFS architecture (NameNode, DataNodes, blocks, replication, rack awareness);

(L3) describe S3's key model, consistency, multipart upload and lifecycle;

(L4) compare striping/erasure coding vs. replication on overhead and recovery;

(L5) reason about metadata scaling (NameNode bottleneck) and its mitigations.

6.1 Three Abstractions

File storage
Hierarchical namespace (directories, files), POSIX-like operations, byte-range reads/writes, strong consistency per file. Examples: NFS, HDFS, GFS.

Block storage
Raw fixed-size blocks addressed by block ID, no files or directories; the OS formats a filesystem on top.

Examples: EBS, Ceph RBD.

Object storage
Flat namespace of variable-size immutable objects addressed by key (bucket/key), rich metadata, HTTP API, massive scale, eventual or read-after-write consistency. Examples: S3, GCS, Azure Blob.

Files suit shared POSIX workloads; blocks suit databases that manage their own layout; objects suit web-scale, write-once, read-many data (images, logs, backups) where scale and durability trump POSIX semantics.

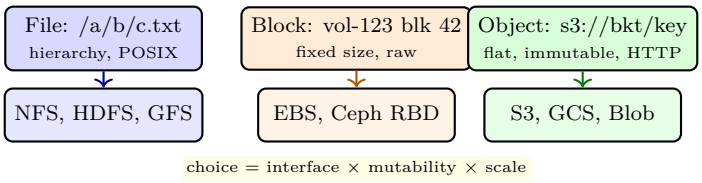

Figure 6.1: Three storage abstractions. Moving right trades POSIX mutability for scale and simplicity.

6.2 HDFS: A Distributed File System

The Hadoop Distributed File System (HDFS) stores very large files as a sequence of fixed-size *blocks* (default 128 MB). Architecture:

NameNode
Holds the namespace, file-to-block mapping and block locations—all in RAM. The single metadata bottleneck.

DataNodes
Store blocks on local disks, report block lists (block reports) and heartbeats to the NameNode.

Client
Asks NameNode for block locations, then reads/writes directly to DataNodes (data path bypasses NameNode).

Replication (default 3×) is rack-aware: replica 1 on the writer's rack, replica 2 on a different rack, replica 3 on the same rack as replica 2—survives one node and one rack failure. Writes are pipelined through the replica chain (as in Ch. 5). HDFS trades POSIX completeness for throughput: write-once, append-possible, no random writes, optimised for streaming reads.

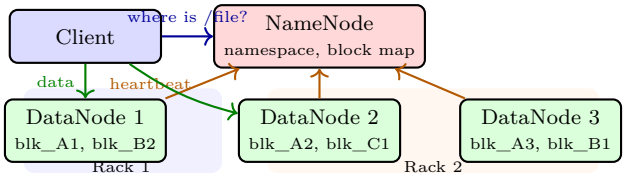

Figure 6.2: HDFS: NameNode holds metadata; clients stream data directly to/from DataNodes. Replication is rack-aware.

NameNode scale. The NameNode keeps ≈ 150 bytes per block in RAM; a 100M-block cluster needs ∼15 GB just for block metadata. Mitigations: HDFS Federation (multiple NameNodes for different namespaces), Na-

meNode HA (active–standby with shared journal), and specialised metadata stores (HopsFS replaces the heap with a NewSQL DB).

6.3 Object Storage and S3

Amazon S3 presents a flat key space: each object lives at `s3://bucket/key` with size up to 5 TB, metadata, and optional versioning. There are no directories—slashes in keys are a console fiction. Objects are immutable: overwrite creates a new version.

Consistency. S3 now provides strong read-after-write for new puts and strong read-after-update/delete (since Dec 2020)—earlier it was eventual for overwrites. Listing remains eventually consistent. Multipart upload splits a large object into parts uploaded in parallel and assembled server-side, enabling resume and high throughput.

Durability and lifecycle. S3 promises 11 nines durability via cross-AZ replication and erasure coding internally. Lifecycle rules transition objects through storage classes (Standard → IA → Glacier) and expire them.

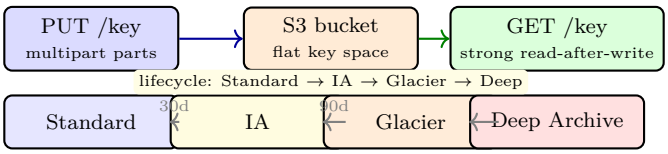

Figure 6.3: S3 object lifecycle and multipart upload. Keys are flat; lifecycle rules automate cost optimisation.

```python
# S3-style multipart upload and HDFS block placement (illustrative)
import hashlib, os

# --- S3 multipart: split, hash each part, reassemble ---
def multipart_split(data: bytes, part_size: int = 5):
    parts = [data[i:i+part_size] for i in range(0, len(data), part_size)]
    etags = [hashlib.md5(p).hexdigest() for p in parts]
    # server reassembles in order; overall ETag is hash of concatenated part hashes
    overall = hashlib.md5(b"".join(bytes.fromhex(e) for e in etags)).hexdigest()
    return list(zip(etags, parts)), overall

data = b"Hello distributed storage world!"
parts, etag = multipart_split(data, part_size=8)
print(f"{len(parts)} parts, overall ETag~{etag[:12]}...")
reassembled = b"".join(p for _, p in parts)
assert reassembled == data

# --- HDFS rack-aware replica placement ---
racks = {"dn1":"rack1", "dn2":"rack1", "dn3":"rack2", "dn4":"rack2", "dn5":"rack3"}
def place_replicas(writer_dn, n=3):
    # replica 1: writer's rack, replica 2: different rack, replica 3: same rack as 2
    writer_rack = racks[writer_dn]
    other_racks = [d for d,r in racks.items() if r != writer_rack]
    replica2 = other_racks[0]
    same_rack_as_2 = [d for d,r in racks.items() if r==racks[replica2] and d!=replica2][0]
```

```
    return [writer_dn, replica2, same_rack_as_2]

print("placement for writer dn1:", place_replicas("dn1"))
```

6.4 Replication vs. Erasure Coding

3× replication costs 200% overhead. *Erasure coding* (e.g., Reed–Solomon k data + m parity) tolerates m failures with overhead m/k. HDFS EC with $6+3$ uses 50% overhead vs. 200% for 3×—a 4× saving. The trade-off is recovery cost: reconstructing a lost block needs k other blocks (more I/O and CPU) and small random reads become expensive.

Hence tiered strategies: hot data stays replicated for fast degraded reads; cold data is EC-encoded for capacity. S3, GFS and Ceph all tier this way.

```
# Erasure coding vs replication: storage overhead and fault tolerance
def overhead(replicas=None, k=None, m=None):
    if replicas: return (replicas - 1) * 100  # percent extra
    return m / k * 100

print(f"3x replication overhead: {overhead(replicas=3):.0f}%")
for k,m in [(6,3),(10,4),(8,2)]:
    print(f"RS({k}+{m}) overhead: {overhead(k=k,m=m):.0f}% tolerates {m} failures")

# Simple XOR parity (k=2,m=1) -- illustrative, not RS
def xor_parity(blocks):
    p = bytearray(len(blocks[0]))
    for b in blocks:
        for i in range(len(p)): p[i] ^= b[i]
    return bytes(p)

a, b = b"\x01\x02\x03", b"\x0A\x0B\x0C"
p = xor_parity([a,b])
recovered_a = xor_parity([b,p])  # a = b xor p
print("XOR recovery:", recovered_a == a)
```

6.5 DHTs: Storage Without a Master

Intuition first: HDFS and S3 both lean on a central metadata authority – the NameNode, the bucket index. A *distributed hash table* (DHT) removes that authority: thousands of peers share one flat key space, and any peer can find the owner of any key without asking a master. The trick has two parts: a ring, and a shortcut.

The hash ring. Hash every node ID and every key into the same m-bit circle (0 to $2^m - 1$). Walk clockwise from a key k; the first node you meet – its *successor* – stores k. When node N4 joins between N3 and N1, only keys in arc $(N3, N4]$ move (to N4); every other key stays put. When a node departs, its arc falls back to its successor. So churn migrates K/n keys on average, versus $K/2$ under naive $h(k) \bmod n$ remapping – this

minimal disruption is *consistent hashing* (Karger et al., 1997).

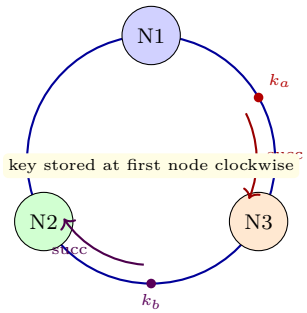

Figure 6.4: Consistent-hash ring. Each key maps clockwise to its successor: $k_a \to$ N3, $k_b \to$ N2. A joining node takes only its own arc; the rest is untouched.

Chord lookup sketch. Naively, a lookup walks the ring successor-by-successor: $O(n)$ hops. Chord (Stoica et al., 2001) gives each node a *finger table* of m entries: finger i points to $\text{succ}(id + 2^{i-1})$, so each hop at least halves the remaining clockwise distance. Worked lookup: $m = 3$ ring (0–7), nodes at $0, 2, 5$; key $k = 6$ asked from node 0. Node 0's best finger short of 6 is 5; node 5's successor is 0 (wrap), which owns arc $(5, 0] \ni 6$. Answer: node 0 in 2 hops, and $O(\log n)$ in general. Kademlia (2002) keeps the same idea but measures closeness by XOR distance and queries the k closest peers in parallel for churn resilience.

DHTs trade strong consistency for decentralisation: joins, departs and lookups are all local protocols, which is why BitTorrent (Mainline DHT), IPFS and Dynamo-style stores build on them.

6.6 Exercises

E6.1 **HDFS block math.** A cluster stores 10 PB in 128 MB blocks with 3× replication. How many blocks and how much NameNode heap (∼150 B/block) is needed? What changes if EC $6 + 3$ is used for 80% of the data?

E6.2 **Rack awareness.** With 3 racks and replication factor 3, why does HDFS place 2 replicas on one rack and 1 on another, rather than 3 distinct racks or all 3 on one rack? Analyse fault tolerance vs. write bandwidth.

E6.3 **S3 consistency.** Before Dec 2020, S3 overwrite was eventual. Give an execution where a writer overwrites key k and a reader immediately reads k and sees stale data, then later sees the new data. How do applications work around this without strong consistency?

E6.4 **EC recovery cost.** For RS(6,3), how many blocks must be read to reconstruct one lost block? Compare degraded-read latency vs. 3× replication. Why is EC unsuitable for small random reads on hot data?

E6.5 **NameNode bottleneck.** A NameNode holds 200M blocks. Estimate heap and GC pressure. Propose two mitigations (federation, off-heap DB) and state what each trades away (namespace isolation, operational complexity).

Chapter Notes

Ghemawat et al., "The Google File System" (SOSP, 2003) and Shvachko et al., "The Hadoop Distributed File System" (MSST, 2010) for GFS/HDFS. S3: Amazon documentation and Corbett et al. for consistency evolution. Erasure coding in storage: Huang et al., "Erasure coding in Windows Azure Storage" (USENIX ATC, 2012); HDFS-EC (HDFS-7285). Ceph: Weil et al. (OSDI, 2006).

Chapter 7

Distributed Transactions: 2PC, 3PC and Spanner

"Transactions are the programmer's way of pretending the world is sequential, even when it is not."
We make that pretence hold across machines.

From zero: from single-machine ACID to distributed commit. We start with what ACID means on one disk, then ask how to commit atomically when participants are distributed and may fail. We build 2PC, see why it blocks, fix it partially with 3PC, then leap to Spanner's TrueTime-based external consistency.

Learning Outcomes

After this chapter you will be able to:

(L1) define ACID and explain which letters distribution threatens;

(L2) trace two-phase commit (2PC) and identify its blocking window;

(L3) describe three-phase commit (3PC) and its extra assumptions;

(L4) explain Spanner's TrueTime, commit-wait and externally consistent reads;

(L5) compare 2PC/Spanner/Percolator on latency, availability and isolation.

7.1 ACID on One Machine, Then Many

On one machine, ACID means: *Atomicity* (all or nothing), *Consistency* (invariants preserved), *Isolation* (concurrent transactions appear serial), *Durability* (committed data survives crashes). The write-ahead log (WAL) plus two-phase locking or MVCC implements this with one disk as the ground truth.

Distribute the data and each letter gets harder. Atomicity now needs *distributed commit*: either all participants commit or all abort, even if some crash mid-protocol. Isolation now spans shards. Durability now means

replicated durability. Consistency remains the application's invariant, but the system must not break it.

A *distributed transaction* touches participants $P_1, \ldots, P_n$ coordinated by a transaction manager (TM). Each participant can vote to commit or abort; the group must agree.

7.2 Two-Phase Commit (2PC)

2PC is the classic blocking atomic-commit protocol:

Phase 1 (Voting). TM sends `PREPARE` to all participants. Each participant that can commit writes a `PREPARED` record to its WAL and replies `YES`; otherwise it writes `ABORT` and replies `NO`.

Phase 2 (Decision). If all voted `YES`, TM writes `COMMIT` and sends `COMMIT`; otherwise it writes `ABORT` and sends `ABORT`. Participants that receive the decision apply it and ack.

If any participant votes `NO` or fails to respond, the transaction aborts. The WAL records make crash-recovery possible: on restart a participant in `PREPARED` state must contact the TM to learn the outcome.

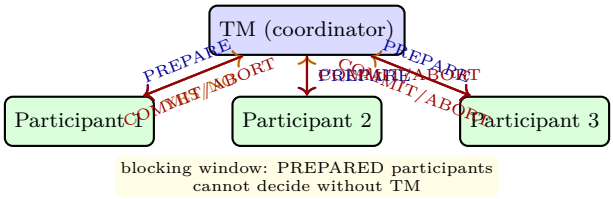

Figure 7.1: Two-phase commit: voting then decision. Participants in PREPARED state are blocked until the TM's decision arrives.

Blocking. If the TM crashes after some participant has entered `PREPARED` but before broadcasting the decision, those participants are *blocked*: they cannot commit (some other participant may have been told to abort) and cannot abort (some may have been told to commit). They must wait for TM recovery. This unavailability is fundamental to 2PC over async networks—it is not a bug but a consequence of needing consensus on the outcome.

```
# 2PC simulation: coordinator + participants with WAL states
class Participant:
    def __init__(self, pid, can_commit=True):
        self.pid, self.can_commit = pid, can_commit
        self.state = "INIT"  # INIT -> PREPARED/ABORTED -> COMMITTED/ABORTED
    def on_prepare(self):
        if self.can_commit:
            self.state = "PREPARED"  # WAL write before reply
            return "YES"
        self.state = "ABORTED"
        return "NO"
    def on_decision(self, decision):
        self.state = decision  # COMMITTED or ABORTED
        return f"P{self.pid}:{self.state}"

```

```
class Coordinator:
    def __init__(self, participants):
        self.participants = participants
        self.state = "INIT"
    def commit(self):
        votes = [p.on_prepare() for p in self.participants]
        print("votes:", votes)
        if all(v=="YES" for v in votes):
            self.state = "COMMIT"
        else:
            self.state = "ABORT"
        # Phase 2: broadcast decision
        for p in self.participants:
            if p.state == "PREPARED":
                print(p.on_decision("COMMITTED" if self.state=="COMMIT" else "ABORTED"))
        return self.state

# Happy path
ps = [Participant(i) for i in range(3)]
print("result:", Coordinator(ps).commit())
# One participant votes NO -> abort
ps2 = [Participant(0), Participant(1, can_commit=False), Participant(2)]
print("result:", Coordinator(ps2).commit())
```

7.3 Three-Phase Commit (3PC) and Non-Blocking Attempts

3PC adds a `PRE-COMMIT` phase between voting and commit so that if a participant is in `PRE-COMMIT`, it knows every other participant voted `YES`. Then, if the TM fails, participants can run a termination protocol among themselves: if any is in `COMMIT`, commit; if any is in `ABORT`, abort; if all are in `PREPARED`, they need to agree—but with the extra phase the ambiguity window shrinks.

Crucially, 3PC is non-blocking *only* under synchrony assumptions (bounded delay + perfect failure detection). Under asynchrony or partitions it either blocks or risks violating safety—which FLP tells us is unavoidable for deterministic consensus. In practice, systems prefer 2PC + a highly available TM (itself replicated via Paxos/Raft) over 3PC.

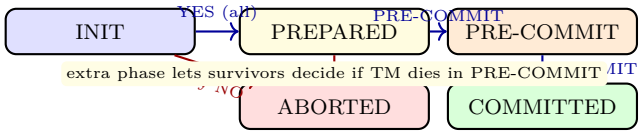

Figure 7.2: 3PC state transitions. The PRE-COMMIT phase makes the protocol non-blocking under synchrony but not under asynchrony.

7.4 Spanner: TrueTime and Externally Consistent Transactions

Spanner (Corbett et al., 2012) is a globally distributed database that provides *externally consistent* (linearizable) read-write transactions with SQL, using TrueTime and 2PC.

TrueTime. Each datacenter has GPS and atomic clocks. TrueTime returns an interval $[earliest, latest]$ guaranteed to contain the true absolute time; typical $\epsilon \approx 2$–7 ms. Spanner assigns each transaction a timestamp ts and enforces *commit-wait*: the coordinator waits until $TT.now().earliest > ts$ before making the write visible. This ensures that if transaction T_1 commits before T_2 starts in real time, $ts(T_1) < ts(T_2)$—so timestamp order respects external order.

Read-write transactions. Spanner runs 2PC across shards, but the prepare phase also picks a prepare timestamp. The coordinator chooses $ts = \max(prepare\_ts) + \text{epsilon}$ and then commit-waits. Paxos-replicated shards provide durability per group. Read-only transactions pick a timestamp in the past and read the snapshot at that timestamp from the nearest replica—lock-free, fast, and still externally consistent.

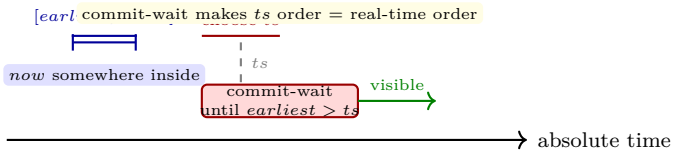

Figure 7.3: Spanner commit-wait: the coordinator picks ts inside the TrueTime interval and waits until TrueTime guarantees ts is in the past before exposing the write.

```python
# Spanner commit-wait intuition with TrueTime intervals
import random

class TrueTime:
    def __init__(self, epsilon_ms=5):
        self.eps = epsilon_ms
    def now(self, real_ms):
        # return [earliest, latest] containing real time
        return real_ms - self.eps, real_ms + self.eps

tt = TrueTime(epsilon_ms=5)
# Transaction picks ts, then waits until earliest > ts
def commit_with_wait(real_pick_ms):
    earliest, latest = tt.now(real_pick_ms)
    ts = latest  # Spanner picks ts within interval, here worst case latest
    # wait until TrueTime.earliest > ts
    wait_until = ts + tt.eps + 1  # earliest = real - eps > ts  => real > ts+eps
    print(f"pick at real={real_pick_ms} interval=[{earliest},{latest}] ts={ts} -> visible
        after real>{wait_until}")
    return ts

t1 = commit_with_wait(100)
t2 = commit_with_wait(120)
print(f"external order: T1 before T2, timestamp order: {t1} < {t2} = {t1 < t2}")

# Read-only transaction: pick timestamp in the past, read snapshot -- no locks
def snapshot_read(read_ts, writes):
    # writes: list of (ts, key, value)
    snapshot = {k: v for ts,k,v in writes if ts <= read_ts}
    return snapshot

writes = [(t1,"x",10),(t2,"x",20)]
print("read at t1 sees:", snapshot_read(t1, writes))
```

```
print("read at t2 sees:", snapshot_read(t2, writes))
```

7.5 Exercises

E7.1 **2PC blocking.** Construct an execution where the TM crashes after sending `PREPARE` and one participant has replied `YES` while another has not yet voted. Show why the `YES` voter is blocked and what information it would need to unblock without the TM.

E7.2 **3PC vs. synchrony.** Explain why 3PC's non-blocking guarantee requires a perfect failure detector (synchrony). Give a partition scenario where 3PC would violate safety if it tried to stay non-blocking under asynchrony.

E7.3 **Spanner timestamps.** Two transactions T_1, T_2 commit with TrueTime $\epsilon = 5\,\text{ms}$. T_1's commit-wait ends at real time 105 ms, T_2 starts at 107 ms. Prove $ts(T_1) < ts(T_2)$. What breaks if commit-wait is skipped?

E7.4 **2PC + Paxos TM.** How does replicating the TM with Paxos/Raft mitigate 2PC's blocking? Describe the recovery path when the old TM leader crashes in `PREPARED` and the new TM leader takes over. What WAL records does it need?

E7.5 **Isolation levels.** Compare Spanner (external consistency), Percolator (snapshot isolation) and 2PC with 2PL (serializable) on (a) read-only latency, (b) write latency, (c) availability under one shard down. Which would you choose for a global ad-auction ledger vs. a social feed?

Chapter Notes

Gray (1978) and Lampson–Sturgis (1979) for 2PC; Skeen (1981) for 3PC and its impossibility without synchrony. Spanner: Corbett et al., "Spanner: Google's globally-distributed database" (OSDI, 2012); TrueTime details in the same paper. Percolator: Peng–Dabek (OSDI, 2010) for snapshot isolation on Bigtable. For a textbook treatment of commit protocols see Bernstein–Hadzilacos–Goodman, *Concurrency Control and Recovery in Database Systems* (1987) and Kleppmann Ch. 9.

Chapter 8

Case Studies: GFS, MapReduce, Kubernetes and Current Research

"You cannot understand a system until you see how it behaves at scale, under failure, and under pressure." We close by tying every earlier chapter to real systems.

From zero: from primitives to platforms. We revisit foundations through four real systems—GFS, MapReduce, Kubernetes—and end with current research directions. Each case study maps to the models, clocks, RPC, consensus, replication, storage and transactions we have built.

Learning Outcomes

After this chapter you will be able to:

(L1) explain GFS architecture and its consistency/availability trade-offs;

(L2) describe MapReduce execution, fault tolerance and why it is not a general computation model;

(L3) outline Kubernetes control-plane (etcd/Raft, controllers, scheduler) and reconciliation;

(L4) map each case study to the earlier chapters (CAP, clocks, consensus, replication);

(L5) identify two current research frontiers (serverless consistency, learned/distributed ML coordination).

8.1 GFS: The Google File System

GFS (Ghemawat et al., 2003) was built for Google's workload: huge sequential reads/writes, append-heavy, thousands of clients, commodity disks that fail often. Design choices reflect those premises (and Ch. 1's fallacies taken seriously).

Architecture. One *master* (metadata: namespace, file → chunk mapping, chunk locations) and many *chunkservers* (store 64 MB chunks, 3× replicated). Clients ask the master for chunk locations, then stream data directly to chunkservers—like HDFS but predating it. The master keeps all metadata in RAM for speed, with an operation log persisted and checkpointed.

Consistency model. GFS offers *relaxed* consistency: multiple writers appending to the same file may interleave, and the offset of an appended record is chosen by the primary chunkserver. Successful mutation is visible atomically at least once (at-least-once append), but failed retried appends can leave duplicates/padding that readers must handle. This is weaker than POSIX but sufficient for MapReduce-style batch pipelines where readers scan and filter.

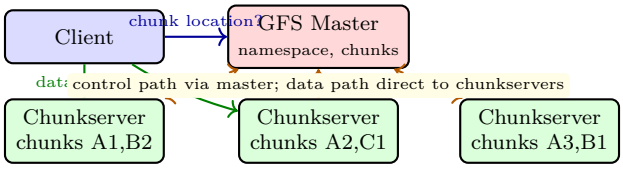

Figure 8.1: GFS: single master for metadata, many chunkservers for data. Data bypasses the master after location lookup.

GFS's single-master weakness (memory, availability) was addressed by Colossus (GFS2) with distributed metadata (Bigtable), and by HDFS Federation. The lesson: metadata scaling (Ch. 6) is often the first bottleneck to hit.

8.2 MapReduce: Data-Parallel Execution

MapReduce (Dean–Ghemawat, 2004) is a programming model plus a fault-tolerant runtime for batch processing over GFS-like storage.

Model. Program two functions: $\texttt{map}(k_1, v_1) \to [(k_2, v_2)]$ and $\texttt{reduce}(k_2, [v_2]) \to [(k_3, v_3)]$. The runtime shards input, runs M map tasks, shuffles intermediate keys to R reducers (grouping by k_2), and writes output back to GFS. The model is deliberately restricted: no iteration, no shared mutable state, no communication between mappers.

Fault tolerance. Map tasks write intermediate output to local disk; on failure the task is re-executed elsewhere (deterministic replay). Reduce tasks read shuffled data; failed reducers re-read from map outputs. The master (coordinator) tracks liveness via heartbeats and reschedules. Stragglers are mitigated by *backup tasks* (speculative execution): run a duplicate of the slowest tasks and take the first result—a direct application of tail-latency handling.

```
# MapReduce word count -- the canonical example (runnable locally)
from collections import defaultdict

def map_fn(doc_id, text):
    for word in text.lower().split():
        yield (word, 1)
```

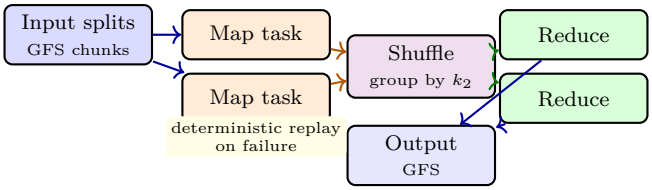

Figure 8.2: MapReduce dataflow: input → map → shuffle (group) → reduce → output. Failures are handled by re-execution; stragglers by backup tasks.

```

def reduce_fn(word, counts):
    yield (word, sum(counts))

# Simulate execution with fault tolerance via re-execution
docs = [(1,"hello world hello"), (2,"world of distributed systems"),
        (3,"hello distributed world")]

# Map phase: each doc is a split; map is deterministic so replay is safe
intermediate = defaultdict(list)
for doc_id, text in docs:
    for k, v in map_fn(doc_id, text):
        intermediate[k].append(v)

# Shuffle is implicit in the dict grouping above
# Reduce phase
output = {}
for word, counts in intermediate.items():
    for k, v in reduce_fn(word, counts):
        output[k] = v

print(sorted(output.items()))
# [('distributed', 2), ('hello', 3), ('of', 1), ('systems', 1), ('world', 3)]

# Straggler mitigation: backup task (first result wins) -- sketch
import concurrent.futures
def slow_map(doc):  # one task is slow
    import time; time.sleep(0.05 if doc[0]==2 else 0)
    return list(map_fn(*doc))

with concurrent.futures.ThreadPoolExecutor() as ex:
    futs = {ex.submit(slow_map, d): d for d in docs}
    for fut in concurrent.futures.as_completed(futs):
        pass  # in real MR, duplicate slowest and take first
```

MapReduce's limitation—no efficient iteration—led to Spark (in-memory RDDs), Flink (streaming) and Dataflow. The core insight remains: restrict the model to make fault tolerance and scheduling tractable.

8.3 Kubernetes: Reconciliation and the Control Plane

Kubernetes orchestrates containers at scale. Its architecture maps directly to earlier chapters: etcd (Raft, Ch. 4) for consistent state, controllers that reconcile desired vs. actual state (a replication idea, Ch. 5), and a scheduler

that places pods.

etcd (Raft). All cluster state (pods, services, config) lives in etcd, a Raft-replicated key-value store (Ch. 4). Every write is linearizable; the AP side is handled by kubelet caching on each node so workloads survive etcd unavailability briefly.

Reconciliation loop. Controllers watch etcd, compare desired state (the spec in etcd) with actual state (what kubelets report), and act to converge them—a continuous control loop, not a one-shot RPC. This makes the system self-healing under Ch. 1's fallacies: messages lost, nodes crash, controllers simply re-reconcile.

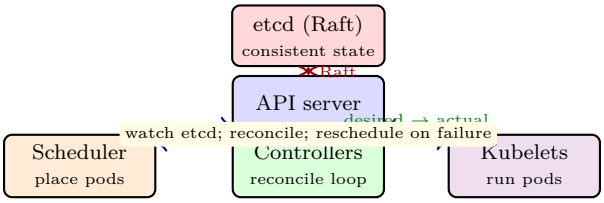

Figure 8.3: Kubernetes control plane: etcd (Raft) stores desired state; controllers and scheduler reconcile actual state toward it.

```python
# Reconciliation loop: desired vs actual state (Kubernetes controller pattern)
import time

desired = {"replicas": 3}   # spec in etcd
actual = {"pods": []}       # reported by kubelets

def reconcile(desired, actual):
    need = desired["replicas"] - len(actual["pods"])
    if need > 0:
        for i in range(need):
            pod = f"pod-{len(actual['pods'])+1}"
            actual["pods"].append(pod)
            print(f"  created {pod}")
    elif need < 0:
        for _ in range(-need):
            pod = actual["pods"].pop()
            print(f"  deleted {pod}")
    return actual

# Simulate: start, scale up, node failure, scale down
for step in ["init", "node failure (lose 1 pod)", "scale to 5", "scale to 2"]:
    print(f"\n[{step}] desired={desired} actual={actual}")
    if step == "node failure (lose 1 pod)":
        actual["pods"].pop()
    if step == "scale to 5": desired["replicas"] = 5
    if step == "scale to 2": desired["replicas"] = 2
    reconcile(desired, actual)
    print(f"  -> actual={actual}")
```

8.4 Current Research Frontiers

Two directions tie the course together:

Serverless and strong consistency. Serverless functions are stateless and short-lived; coordinating them with strong guarantees (exactly-once workflows, distributed transactions across functions) is an active area (Beldi, Olive, Cloudburst). The tension is Ch. 2–4 at the extreme: no stable process to hold a lock, yet the application wants transactions.

ML coordination at scale. Training large models needs parameter synchronisation across thousands of GPUs. Choices mirror Ch. 5: synchronous all-reduce (BSP, strong consistency, straggler-sensitive) vs. asynchronous parameter servers (stale gradients, faster but noisy) vs. bounded-staleness (SSP). Emerging work uses in-network aggregation and RDMA to push replication into the fabric itself.

Both frontiers ask the same question the course began with: what guarantees can we provide when the network is unreliable, clocks diverge and machines fail independently? The primitives—quorums, vector clocks, consensus, commit protocols—reappear, just at new scales and with new hardware.

8.5 Exercises

E8.1 **GFS consistency.** Two clients concurrently append to the same GFS file. Explain why the file may contain interleaved records and duplicates after retries. How does MapReduce tolerate this when reading GFS output? Propose a record format that makes duplicates detectable.

E8.2 **MapReduce stragglers.** A job has 1000 map tasks; 5 are 10× slower (stragglers). Without backup tasks the job takes T. With backup tasks that duplicate the slowest 10 tasks after 80% completion, estimate the speedup. When does speculation hurt (cost vs. benefit)?

E8.3 **Kubernetes and Raft.** etcd uses Raft with 5 members. How many failures can it tolerate for reads and writes? If etcd is partitioned 3–2, which side serves linearizable writes? How do kubelets keep pods running during the partition?

E8.4 **Putting it together.** Map each case study (GFS, MapReduce, Kubernetes) to chapters 1–7: identify the failure model, timing assumption, consistency level, replication strategy and coordination mechanism it uses. Present as a table.

E8.5 **Research design.** Design a serverless workflow that must atomically update two Dynamo-style stores (eventual). Propose a solution using idempotency keys plus a transaction coordinator (2PC-like). Analyse what happens if the coordinator crashes mid-workflow and how you would make it highly available.

Chapter Notes

GFS: Ghemawat–Gobioff–Leung (SOSP, 2003); Colossus/Bigtable evolution in Corbett et al. (2012). MapReduce: Dean–Ghemawat (OSDI, 2004); Spark: Zaharia et al. (NSDI, 2012); Dataflow: Akidau et al. (VLDB, 2015). Kubernetes: Burns et al., *Borg, Omega, and Kubernetes* (CACM, 2016); etcd/Raft documentation. Serverless consistency: Zhang et al., "Cloudburst" (VLDB, 2020); Beldi (OSDI, 2020). ML coordination: Li et al., "Scaling distributed ML with parameter server" (OSDI, 2014); SSP: Ho et al. (2013).